Analyse syntaxique ascendante : LR(0) et SLR(1)

Compilation – M1 Informatique – Fiche de révision

1. Principe de l'analyse ascendante (LR)

- **LR** = lecture de gauche (**L**eft) à droite, dérivations droites (**R**ight).
- **Décaler** : lire un terminal du mot. **Réduire** : remplacer une partie droite de règle par sa partie gauche.
- L'analyse réussit si le mot courant final est l'axiome S.
- Une **table d'analyse** indique s'il faut décaler ou réduire (et par quelle règle).

2. Construction d'une table LR(0)

Étapes : (1) grammaire augmentée, (2) automate des items, (3) tables Action et Successeur.

- **Grammaire augmentée** : nouvel axiome S' et règle S' → S. Un nœud S' marque la racine de l'arbre.
- **Item** (règle pointée) X → β.γ : on a lu β, on espère lire γ. Item **complet** si γ = ε (point à la fin).
- **Fermeture** : si X → β.Yγ est dans l'état et Y non terminal, ajouter Y → .α pour chaque règle Y → α.
- **État initial** : Fermeture({S' → .S}).
- **Transition** Δ(q, x) = Fermeture({X → βx.γ | X → β.xγ ∈ q}). On répète jusqu'à ne plus créer d'état.

3. Tables Action et Successeur

Table	Lignes	Colonnes	Contenu
Action	états	terminaux et $	**d q'** (décaler, aller en q'), **X → β** (réduire), **accepter**, case vide = erreur
Successeur	états	non-terminaux	état d'arrivée Δ(q, X)

Remplissage de Action en LR(0) :

- Si Δ(q, a) = q' : mettre **d q'** en Action[q, a].
- Si q contient X → β. (X ≠ S') : mettre **X → β** dans **toute la ligne** (tout a ∈ T ∪ {$}).
- Mettre **accepter** en Action[q, $] pour l'état contenant S' → S.

4. Grammaire LR(0)

Pour tout état q de l'automate :

- pas de **conflit réduction/réduction** : au plus un item complet par état ;
- pas de **conflit décalage/réduction** : si q contient un item complet, aucune transition par un terminal ne part de q.
- *Exemple LR(0)* : S → Ac, A → AaAb | d.
- *Contre-exemple* : E → E+T | T, T → T*F | F, F → nb. Les états 4 (E → T. et T → T.*F) et 8 (E → E+T. et T → T.*F) ont un conflit décalage/réduction.

5. SLR(1)

Idée : ne réduire que si c'est plausible, en utilisant **Suivants(X)**. C'est la seule différence avec LR(0).

	LR(0)	SLR(1)
Décalages	d q' si Δ(q, a) = q'	identique
Réduction X → β	dans **toute la ligne**	seulement pour a ∈ **Suivants(X)**
accepter / Successeur	identiques	identiques
Lecture en avance	0 symbole	1 symbole (via Suivants)

- **Grammaire SLR(1)** : chaque case de la table Action contient **au plus une action**.

- *Exemple* : E → E+T | T ; T → T*F | F ; F → nb. Suivants(E) = {$, +}, Suivants(T) = {$, +, *}. À l'état 4, on réduit E → T seulement sur + et $, et on décale (d6) sur *. Plus de conflit : la grammaire est SLR(1).

6. Exécution de l'analyseur

La pile alterne symboles et états (départ : **$ 0**). Soit qm l'état au sommet et a le prochain symbole :

- **d q'** : empiler a puis q', avancer dans le mot.
- **X → β** : dépiler 2×|β| éléments (symboles et états), puis empiler X et l'état Successeur[état découvert, X].
- **accepter** : succès. **Case vide** : échec.
- **Arbre de dérivation** : lire les réductions du bas vers le haut dans l'analyseur, les construire de la droite vers la gauche sur l'arbre.

7. Grammaires ambiguës et LR(k)

- Grammaire E → E+E | E*E | (E) | nb : **4 conflits décalage/réduction** (états 8 et 9, colonnes + et *). Elle n'est pas SLR(1).
- **Théorème** : une grammaire ambiguë n'est pas SLR(1).

Conflit	Choix	Règle
E+E.+E	réduire : [E+E]+E	+ associatif à gauche
E+E.*E	décaler : E+[E*E]	* prioritaire sur +
E*E.+E	réduire : [E*E]+E	* prioritaire sur +
E*E.*E	réduire : [E*E]*E	* associatif à gauche

- **Solution** : grammaire non ambiguë (E → E+T | E-T | T ; T → T*F | T/F | F ; F → (E) | nb). Associativité à gauche = récursivité à gauche ; plus un opérateur est prioritaire, plus il est traité profondément dans la grammaire.
- **LR(k)** : on enrichit les items avec k symboles de prévision, ce qui résout certains conflits SLR(1).

8. Hiérarchie des grammaires

LR(0) ⊂ SLR(1) ⊂ LR(1) ⊂ grammaires non ambiguës ⊂ toutes les grammaires.

LL(1) ⊂ LR(1), mais LL(1) n'est pas inclus dans SLR(1).